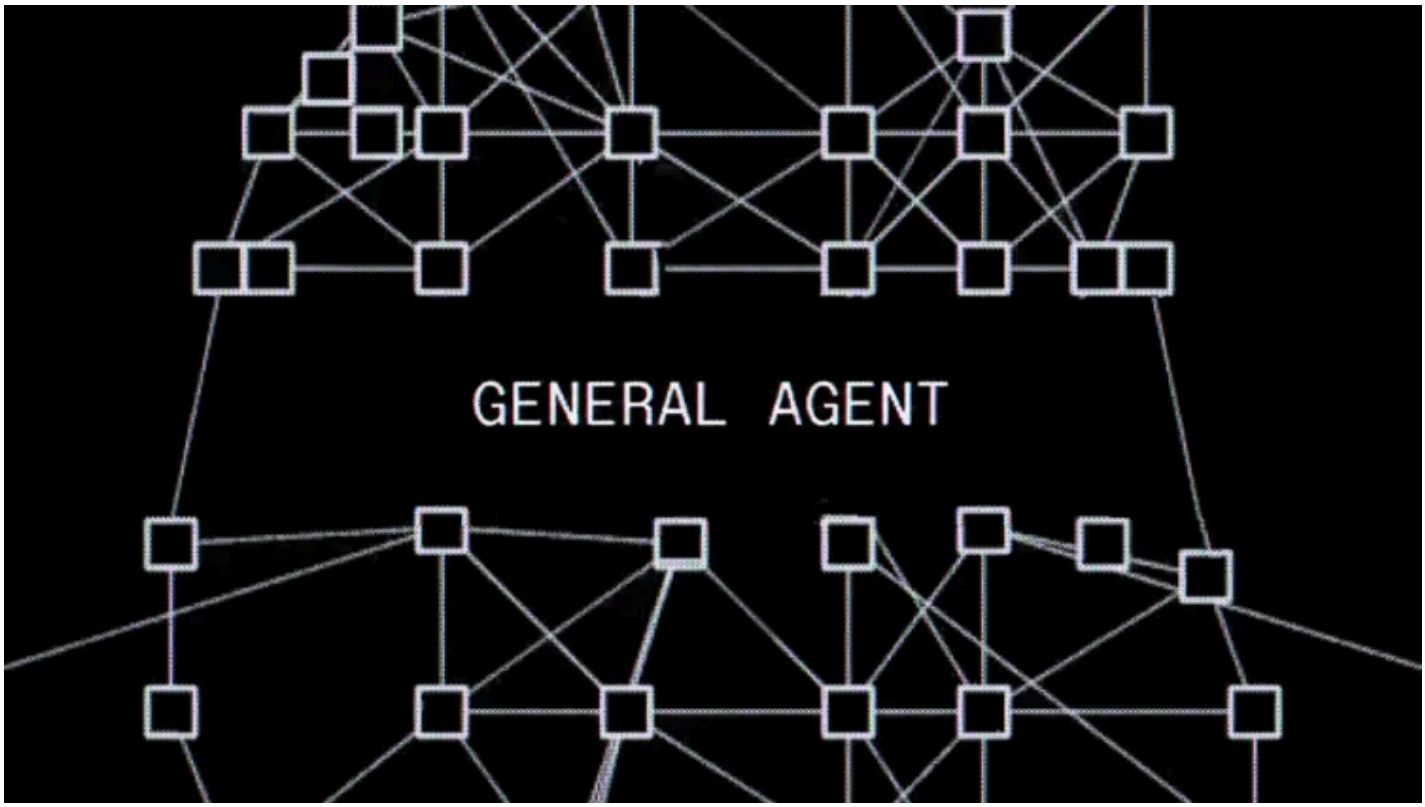


Authors

Mika

Research

May 18, 2026

General Agent: A Self-Evolving, Synthetic Agent Environment

Training capable agents requires exposure to diverse tasks and tools throughout the whole post-training pipeline. Yet, agentic environments with exposure to 1000s of tools remain scarce in the open-source community.

Today, we are open-sourcing a first version of the `general-agent` environment (Environments Hub) — a fully synthetic environment capable of growing its task corpus to be more diverse and challenging over time. It formulates synthetic task creation as a 2-player game between two agents:

diversity, and difficulty.

- **Solver** — An agent tasked to solve a task instance.

The synthesizer agent designs and evolves a novel task family in difficulty tiers. Each tier is empirically validated by running a solver against it — only tasks whose pass rate lands in a calibrated difficulty band are accepted. Hard tiers seed the next wave of extensions, letting the corpus grow progressively harder over time.

The result is a self-evolving task corpus of currently **4,504 tasks** across **1,040 domains** with over **8,000 unique tools**, all grounded in stateful database operations with semantic verification.

Task Anatomy

Every task follows a clear semantic structure: a *database*, a set of *tools* that manipulate the database, an *instruction*, and a *gold solution* with matching *verification function* that checks whether the task was completed successfully.

Task Seed: DB + Tools + Tasks + Verification

Each task defines a Pydantic data model (`DB`) representing entities and their interactions. For example, in the `day_spa` task family, a `Therapist` performs a `Service` during an `Appointment` .

The agent interacts with this database through tools (`Tools`), which are simple Python functions reading or manipulating the state of the database. Tools enforce domain logic (specialty matching, availability checks, rating constraints) and return structured feedback to the agent about data manipulation. For example, the agent may `list_services` , `list_therapists` , and `book_appointment` .

```
1 class Service(BaseModel):
```

```python
    duration_minutes: int
    price: float

class Therapist(BaseModel):
    id: str
    name: str
    specialties: list[str]  # service categories they can perform
    is_available: bool = True

class Appointment(BaseModel):
    id: str
    customer_name: str
    service_id: str
    therapist_id: str
    status: str = "booked"

class TaskDB(DB):
    services: list[Service] = []
    therapists: list[Therapist] = []
    appointments: list[Appointment] = []
    target_customer: str | None = None
    target_service: str | None = None

class TaskTools(Tools):
    db: TaskDB

    @tool
    def list_services(self) -> list[dict]:
        """Return all spa services with details."""
        return [s.model_dump() for s in self.db.services]

    @tool
    def list_therapists(self) -> list[dict]:
        """Return all therapists with their specialties."""
        return [t.model_dump() for t in self.db.therapists]

    @tool
    def book_appointment(self, appointment_id: str,
                         customer_name: str, service_id: str,
                         therapist_id: str) -> dict:
        """Book a spa appointment."""
        service = next(s for s in self.db.services if s.id == service_id)
        therapist = next(t for t in self.db.therapists if t.id == therapist_id)
        # check therapist availability and specialty match
        # mark therapist unavailable, append appointment
        ...
```

complete the task. For the seed task `day_spa_t0`, the instruction reads

> Hi, I'm Sarah and I'd like to book a Swedish Massage. Could you look up the services and therapists, then just go ahead and book me with any available therapist who does massage? Don't worry about asking me to pick, just pick one and book it.

To ensure that tasks are solvable, the synthesizer is asked to produce a *gold solution* and a *verification function*. It is expected that the verification function checks all the constraints listed in the instruction, and the synthesizer has to produce a gold solution which passes its own verification function. For the simple seed task `day_spa_t0`, the `verify` function simply checks that the target customer has a booked appointment for the target service.

```python
def verify(db: TaskDB) -> float:
    """Check that the target customer has a booked appointment
    for the target service."""
    if not db.target_customer or not db.target_service:
        return 0.0
    for a in db.appointments:
        if (a.customer_name == db.target_customer
        and a.service_id == db.target_service
        and a.status == "booked"):
            return 1.0
    return 0.0
```

The gold solution the synthesizer provides for this example requires listing all services and therapists to find an available massage therapist, then booking the appointment.

```
[
  ["list_services", {}],
  ["list_therapists", {}],
  ["book_appointment", {"appointment_id": "A1", "customer_name": "Sarah", "service_id":
]
```

For initial validation of the task, we simply require that replaying the gold solution changes the DB state such that verification flips from failing to passing. It checks:

This test is a simple structural soundness check: if this check fails, something about the task is fundamentally broken. It serves as a first sanity check for task solvability when the synthesizer creates a new task instance.

Task Evolution: Difficulty Tiers

Following DeepSeek-V3.2's methodology (arxiv:2512.02556), we find that iteratively evolving a task from trivial to challenging is substantially easier than one-shotting a challenging task. Hence, the synthesizer first constructs a simple seed task following the spec outlined above and then iteratively increases its difficulty by extending the instruction, DB, and tools at each step.

Currently, each task family spans **5 difficulty tiers** (`t0` through `t4`), where higher tiers grow from lower tiers by applying one or more of the following evolution strategies.

Method	What it does	Example
`multi_step_reasoning`	Answer requires combining results from multiple tool calls	Combine `search_services` to find sensitive facial + `list_therapists` to filter by rating ≥ 4.7 before booking
`conditional_rules`	Conditional constraints that require branching logic	If skin is sensitive, facial must have `skin_type: sensitive`
`cross_entity_coupling`	No repeats, sum constraints, dependency chains, mutual exclusivity across entities	No therapist may be reused across multiple bookings
`stricter_thresholds`	Budget limits, rating minimums, capacity caps, or other numerical constraints	Total cost should be ≤$195
`larger_db`	More entities to search through and more distractors; forces filtering over large datasets	DB grows from 26 to 97 entities

`schema_extension`	Add new DB entity types and relationships, expanding the data model	New `Package` and `Product` entities
`tool_proliferation`	Add plausible-looking but irrelevant distractor tools	Added tools (`list_packages` , `list_products`) which are not needed in the gold solution
`noisy_instructions`	Realistic typos, misspellings, or grammatical errors (2–5 per instruction)	"treaments", "thas", "dont", "skins been acting up"
`ambiguity_resolution`	Instruction requires the agent to disambiguate via tool calls, or tools have less informative names	"Prenatal Massage" looks contextually perfect for a pregnant client but has `pressure: medium` — agent must check the data field, not the name

We ensure that every family uses at least 5 unique evolution strategies across its tiers. Importantly, each tier is empirically gated against a target pass-rate band of a specified solver model. This ensures that the synthesizer produces solvable and sufficiently difficult tasks for each tier.

The `day_spa` family illustrates this evolution: The number of available tools, DB size, and gold steps required to solve the task all increase monotonically, resulting in a clean inverse relationship between the average solve rate and the difficulty tier. Since we used GPT-5-Mini for difficulty calibration, its solve rates fall exactly within the target bands. We can see that the difficulty ladder generalizes to stronger models such as GLM-5.1 which also has lower solve rates in higher difficulty tiers, despite not having been explicitly used for difficulty calibration.

Tier	Target Solve Rate	GPT-5-mini Solve Rate (Avg@50)	GLM-5.1 Solve Rate (Avg
t0	1.0-0.8	1.00	1.00
t1	0.8-0.6	0.75	1.00
t2	0.6-0.4	0.55	0.66

t3	0.4-0.2	0.35	0.10
t4	0.2-0.0	0.20	0.22

To understand where models actually fail, we analyzed GLM's solve attempts and found common failure modes resulting from the increased task difficulty.

`day_spa_t2`

This tier introduces three new evolution strategies (`larger_db` , `cross_entity_coupling` , `stricter_thresholds`) and grows the database from 26 to 97 entities. The instruction now asks for three services — a sensitive-skin facial, a gentle massage, and a manicure — under a $195 budget, with no therapist or room reused across bookings. GLM-5.1 solves this 66% of the time.

> I'm Olivia and I want to treat myself to a full spa day. I've got sensitive skin that's pretty reactive, so I need a facial that's specifically made for that — not something generic. I also want a gentle massage, nothing too hard on the pressure. And I'd love to get my nails done too. My total budget is $200.

In almost all failed rollouts, the model picks a massage with medium pressure instead of a light-pressure option. The particular massage the model picks is known for gentle flowing motions, so the model wrongly reasons it qualifies as "gentle". The model systematically substitutes world knowledge for what's in the database.

`day_spa_t4`

This tier applies three evolution strategies (`ambiguity_resolution` , `cross_entity_coupling` , `noisy_instructions`), growing the database to 184 entities. The instruction is littered with misspellings and introduces a pregnancy constraint. GLM-5.1 solves this just 22% of the time.

> Hey so I'm Mia and I'm expecting a baby in a few months, gotta be careful with treaments. My skins been acting up like crazy, super reactive, so I need a facial thas specifically for

simple nail thing done too. Got $210 to spend total and I only trust therapists rated 4.7 or better. Just find what works and book it for me, dont need to run it by me first.

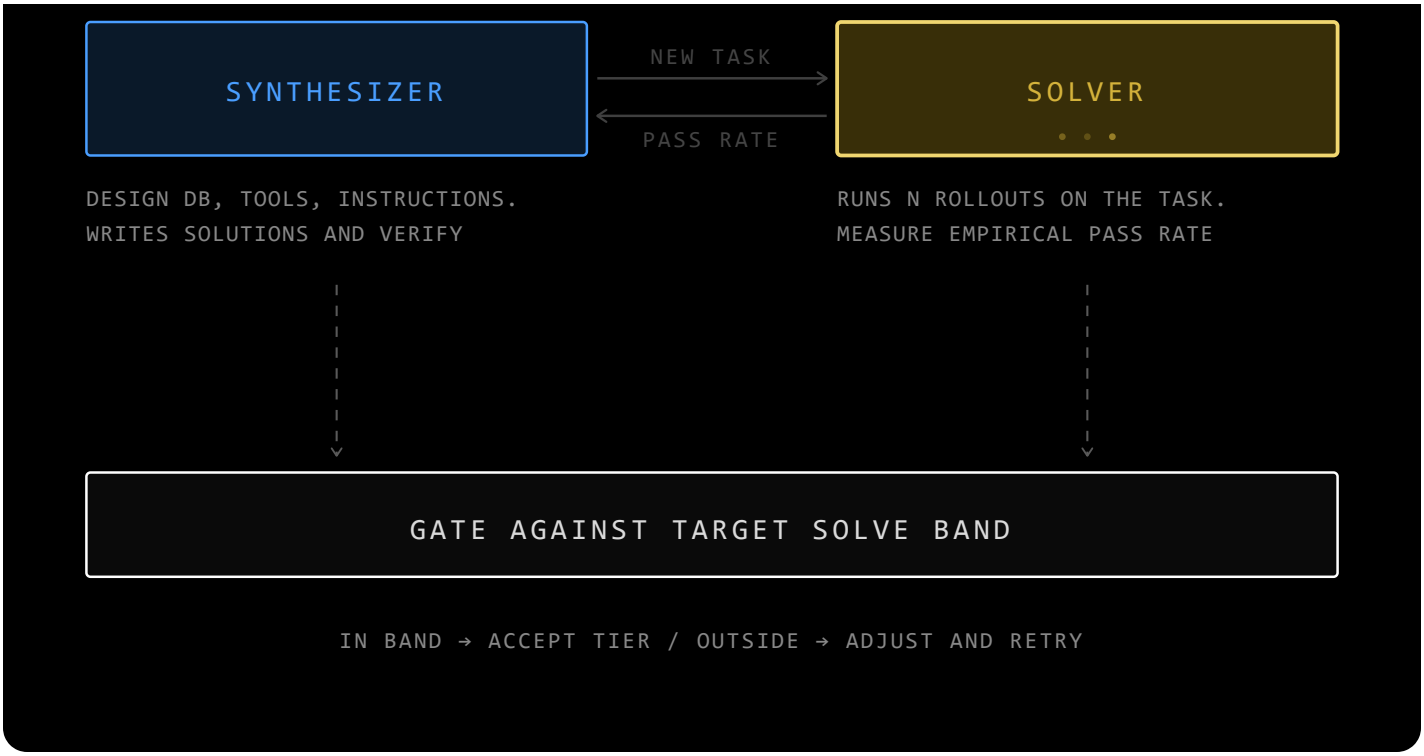
A service called "Prenatal Massage" appears — designed for pregnant clients, contextually perfect. But its pressure field is medium, not light. In most failures, the model books it. The correct answer is the Couples Massage ($87, with light pressure) — a less obvious pick, but the one that matches the constraint. Budget violations further compound into more failed than successful solve attempts.

Task Synthesis

The environment is built around two types of agents that work together in a feedback loop to generate and evolve tasks:

- **Synthesizer** — an agent that designs new task families and evolves them through difficulty tiers. It was used *offline* to generate the task corpus in a 2-player game with a solver.
- **Solver** — an agent that attempts to solve task instances. It serves two roles: as the gating model during synthesis (to calibrate difficulty), and as the optimization target during RL training.

Both of these agents are implemented as `verifiers` environments, which lets them plug natively into the Prime Intellect ecosystem. This essentially brings infrastructure for evaluating, training, and running synthetic data generation at massive scale for free. For example, to generate the initial task corpus **we ran over 1,000 synthesizing GLM-5.1 agents in parallel over multiple days with barely any supervision**.



The two-player loop: the synthesizer proposes tasks, the solver estimates pass rate, and the gate accepts or retries each tier.

Synthesizer

The core insight is that **task creation is itself an agent task**. The synthesizer is an LLM agent running in a sandboxed environment with access to the `general-agent` CLI. It is guided by a structured skill that defines the task format, evolution strategies, gating criteria, and the full synthesis protocol. The skill ensures consistency across the corpus while giving the agent creative freedom in choosing domains and designing tasks and constraints. Each synthesis follows the following steps:

1. **Design** — Pick a novel domain, design a DB schema, and define the tool API.
2. **Seed** — Write the simplest useful task in the domain. Write a verification function and produce a passing gold solution.
3. **Gate** — Run the solver against the seed tier with 20 rollouts. If the solve rate is ≥0.80, the seed is accepted. If not, adjust and retry.

for 20-40% solve rate for `t3`).

5. **Validate** — Final check: the family must use ≥5 unique evolution strategies across its tiers.

This loop ensures that every task in the corpus is:

- **Structurally valid** — gold solution replays correctly, verify function agrees
- **Empirically calibrated** — difficulty isn't guessed, it's measured
- **Diverse** — each family uses multiple independent evolution strategies

Solver

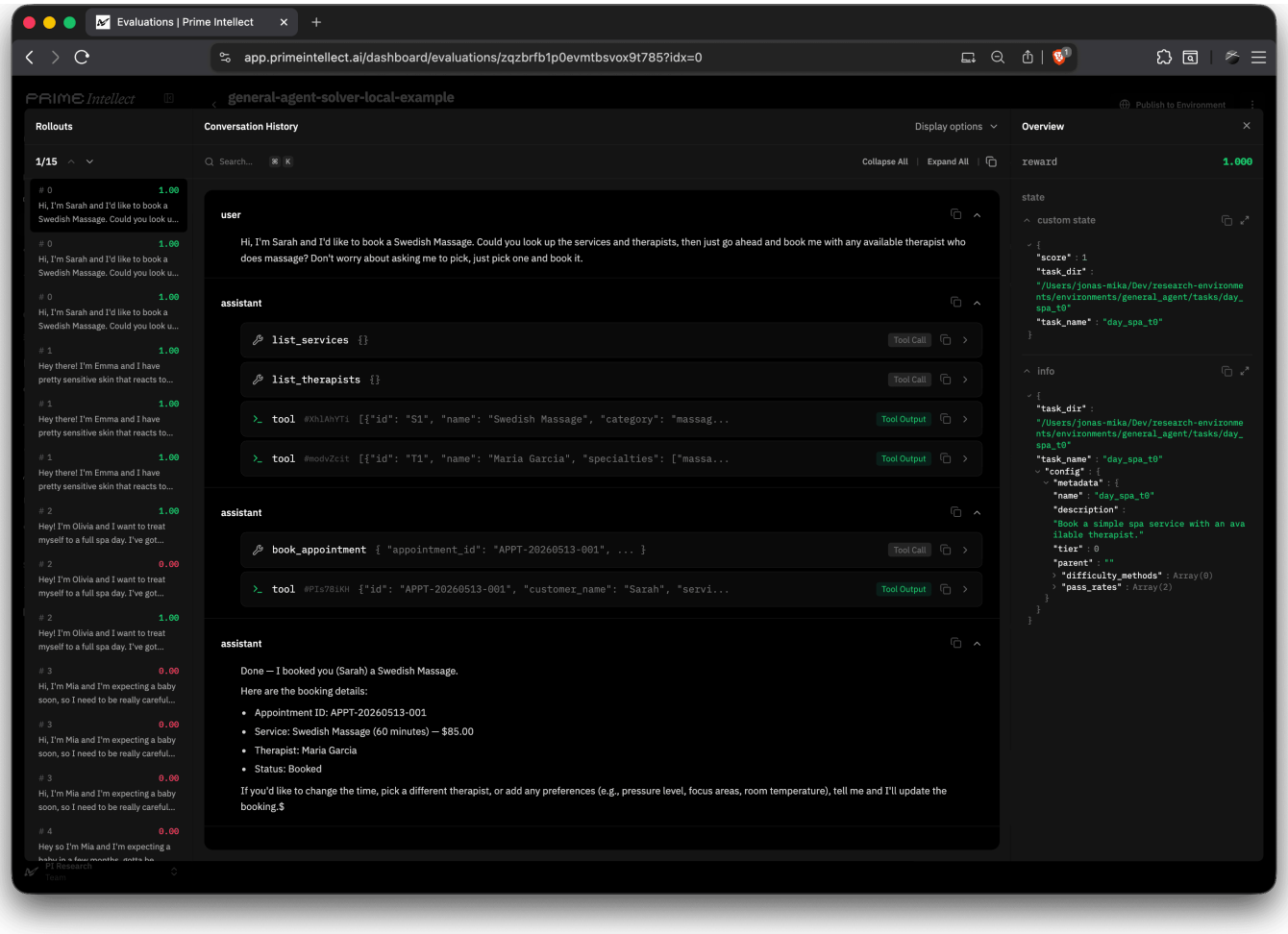
All three backends use the same task format and scoring — they differ in how the agent interfaces with the tools.

1. **Local** — Calls tools directly as Python functions in-process. Used for fast iteration and as the pass-rate gating engine inside the synthesizer's sandbox.
2. **OpenCode** — Runs an OpenCode agent in a sandbox. Task tools are exposed via a local MCP server. Each tool method becomes a native MCP tool the agent can call.
3. **RLM** — Runs an RLM agent in a sandbox with per-tool skills. Each `@tool` method is wrapped in an RLM skill that the agent invokes from its IPython kernel via `await <tool>.run(...)`.

For example, we can generate 5x3 rollouts to solve the `day_spa` task family with `gpt-5-mini` as a local solver using the following command.

```
# Local solver
prime eval run general-agent-solver-local -a '{"task": "day_spa"}' -m openai/gpt-5-mini
```

All trajectories are uploaded to Prime Lab. Below, we can see the model solve the seed task using 2 parallel tool calls to check for the available services and therapists, followed by a single tool call



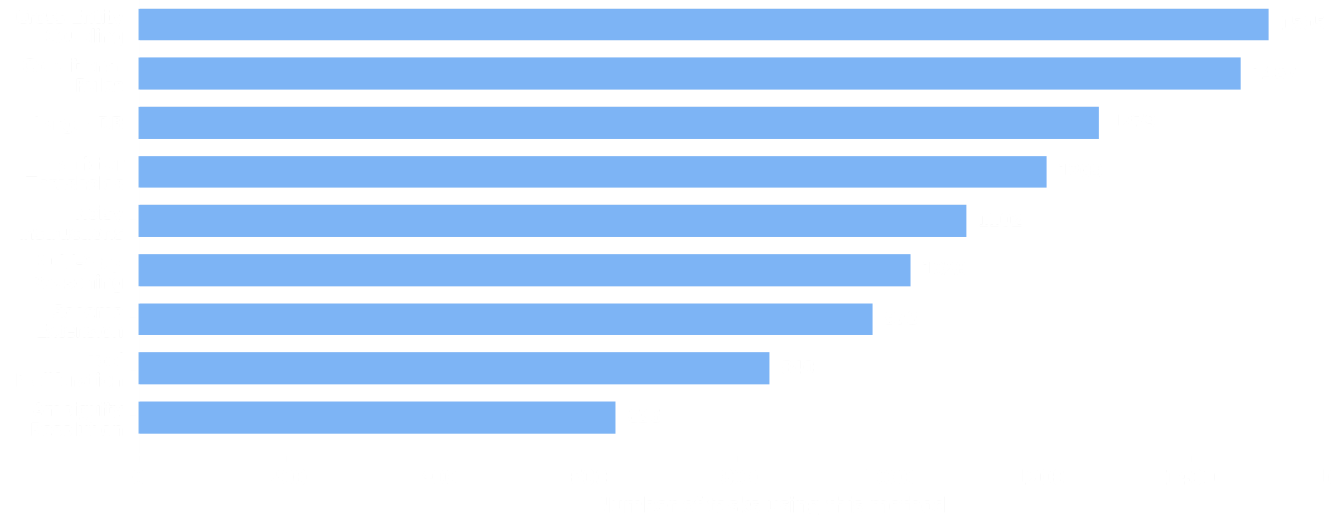
Example Prime Lab rollout where the solver completes the seed task with parallel tool calls.

Task Corpus

The synthesis loop produced **4,504 tasks** across **1,040 domains** using `zai-org/GLM-5.1-FP8` as the synthesizer and `openai/gpt-5-mini` as the solver model for gating at avg@20. Each family is a self-contained world with its own DB schema, logic, and verification criteria. Every task has been empirically measured against at least one solver model. Further, we generated 200K+ traces using `zai-org/GLM-5.1-FP8` as the solver model in the RLM harness to get robust estimates of the task difficulty from a much stronger model.

(Pydantic schema types like `Customer`, `Booking`, `AirspaceZone`). 78% of tools and 66% of entity classes are unique to a single family — the rest are shared abstractions (`Order`, `Equipment`, `get_customer`) that recur across domains.

All 9 evolution strategies are well-represented across the corpus. `cross_entity_coupling` and `conditional_rules` are the most frequent, while `ambiguity_resolution` is the rarest — it requires more careful task design to introduce genuine disambiguation challenges.



Evolution strategy usage across synthesized tasks.

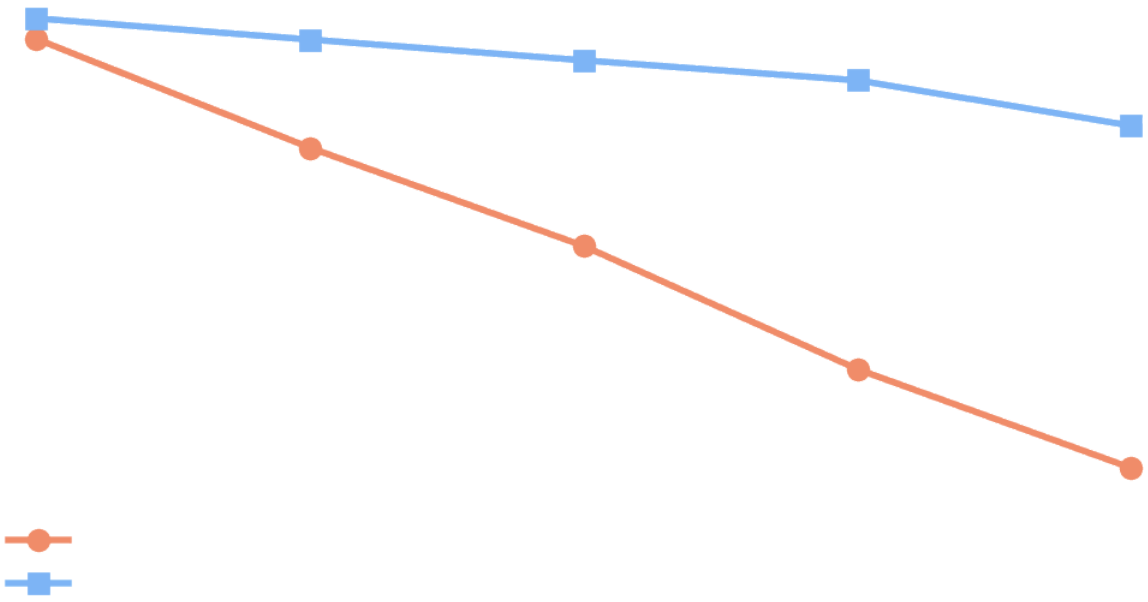
Difficulty Calibration

The corpus was calibrated using GPT-5-mini as the gating solver. The table below shows corpus-wide averages per tier — solve rates, tool counts, gold steps, and DB size all scale monotonically with tier.

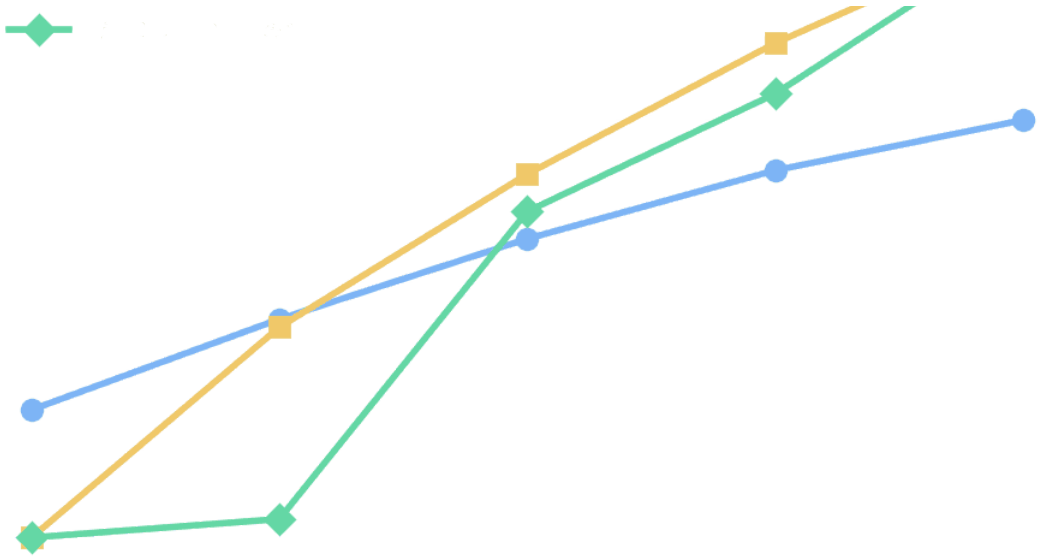
Tier	GPT-5-mini avg@20	GLM-5.1 avg@50	#Tools	#Gold steps	#DB entities
t0	0.928	0.961	6.3	2.5	10
t1	0.757	0.928	9.0	8.7	23
t2	0.601	0.895	11.4	13.3	240

t3	0.407	0.865	13.4	17.2	323
t4	0.251	0.792	14.9	20.5	437

GPT-5-Mini’s solve rates fall within the target bands by design (it was the gating model). The difficulty ladder generalizes to GLM-5.1, which also shows decreasing solve rates across tiers — though with a flatter slope as it’s a stronger model.

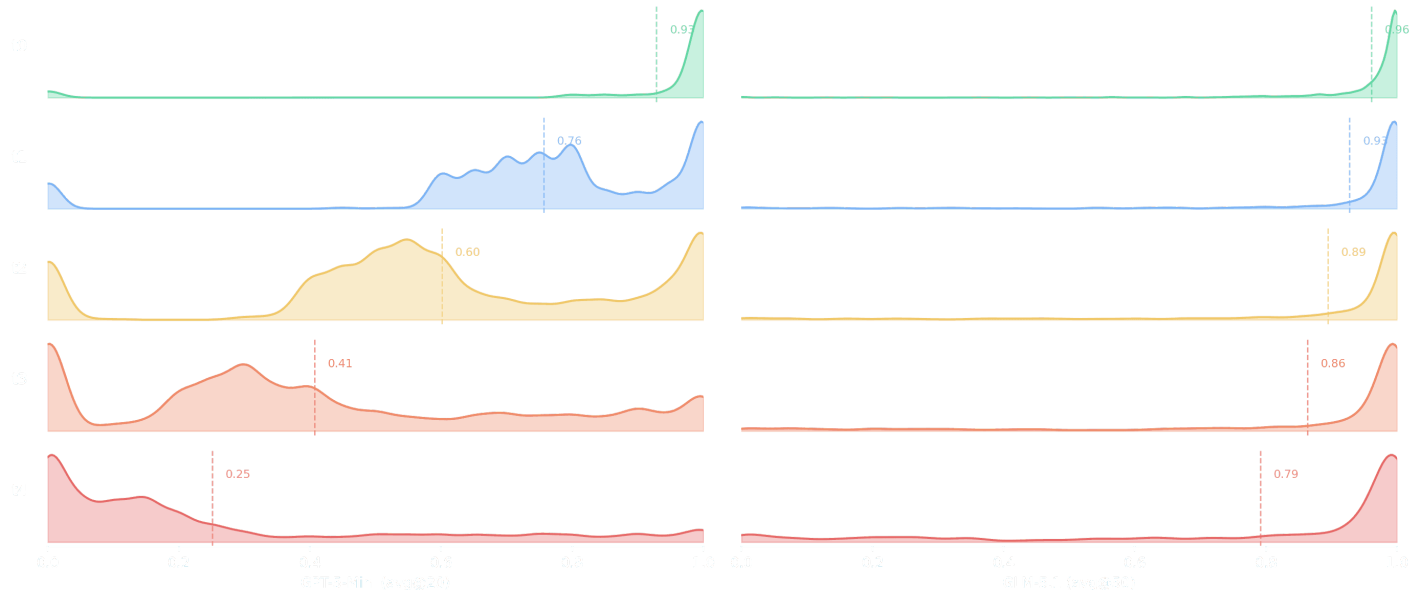


> Solve rate by tier. Both models' solve rates fall monotonically with difficulty; GLM-5.1, as a stronger model, has a gentler decline and achieves high pass rates even in the most difficult tier.



Tools, gold steps, and DB entries by tier. Proxies for task difficulty increase monotonically with difficulty.

Below we break down the average solve rate per tier and per model into distributional plots. For GPT-5-Mini (left), the distribution shifts cleanly from right to left: at `t0` nearly all tasks are solved, by `t4` the mass concentrates below 0.3. For GLM-5.1 (right), the distribution stays right-skewed even at `t4` (median 0.98), reflecting a much stronger model. However, the left tail grows with each tier — the fraction of tasks where GLM fails increases monotonically, confirming that the difficulty ladder generalizes beyond the model it was calibrated on.



However, this plot reveals that the potential to improve GLM-5.1 on this particular version of the dataset is relatively low. Improving tool calling performance on stronger models would likely require further evolving the task corpus using the currently hardest `t4` tier as a seed and gating against strong OSS models.

Early Training Results

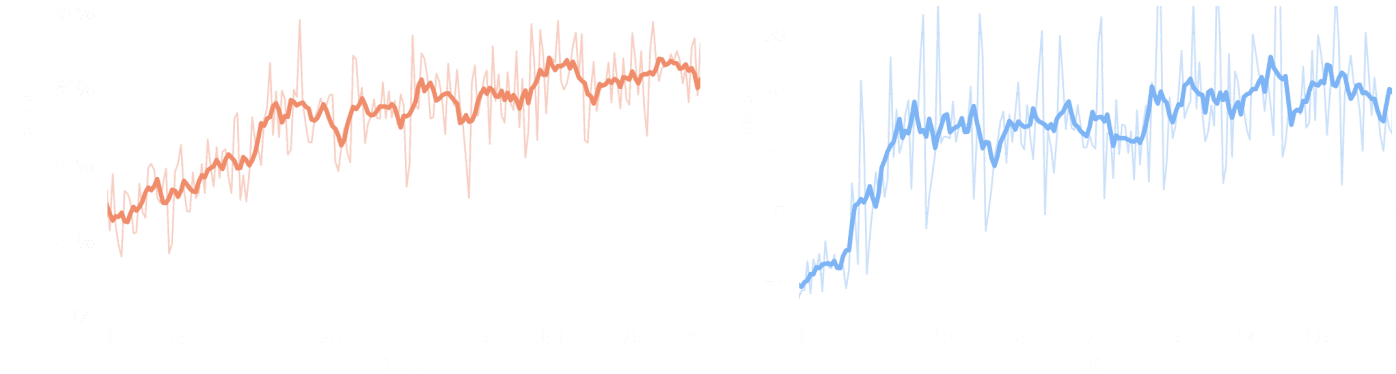
The ultimate goal of the `general-agent` environment is to improve a model's tool-calling and agentic ability. To stress-test the environment, we conducted two simple SFT and RL experiments and evaluated against established benchmarks including BFCL (Berkeley Function Calling Leaderboard, which tests function calling accuracy across diverse real-world tool APIs), and a subset of MCP-Atlas (which benchmarks multi-step tool orchestration against real MCP servers).

RL

First, we showcase that the environment is trainable via RL. We ran a small RL run on `Qwen/Qwen3-30B-A3B-Instruct` on the entire task corpus, excluding the trivial `t0` seed tasks.

Setup. We train for 200 steps with a constant learning rate of 1e-6, 32×16 rollouts per batch and 32k token sequence length.

Results. Average reward climbs from 30% to ~70% over the course of training, with the steepest gains in the first 100 steps. The average number of turns per rollout increases from ~8 to ~24, showing the model learns to make more tool calls and improve in reliably solving its training tasks.



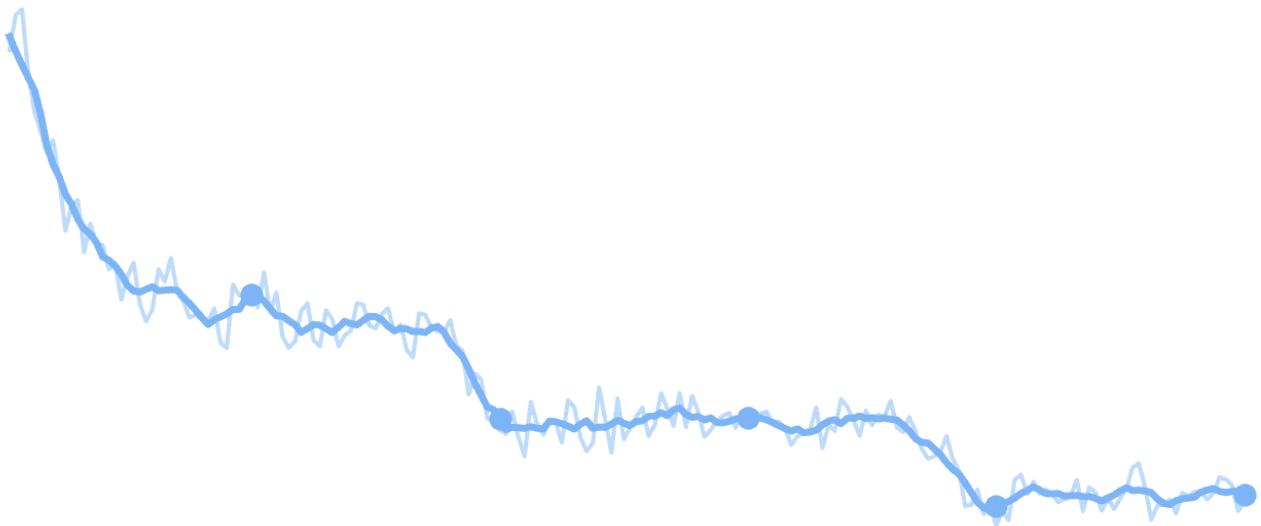
RL training. Both reward and number of turns per rollout increase over 200 training steps.

SFT

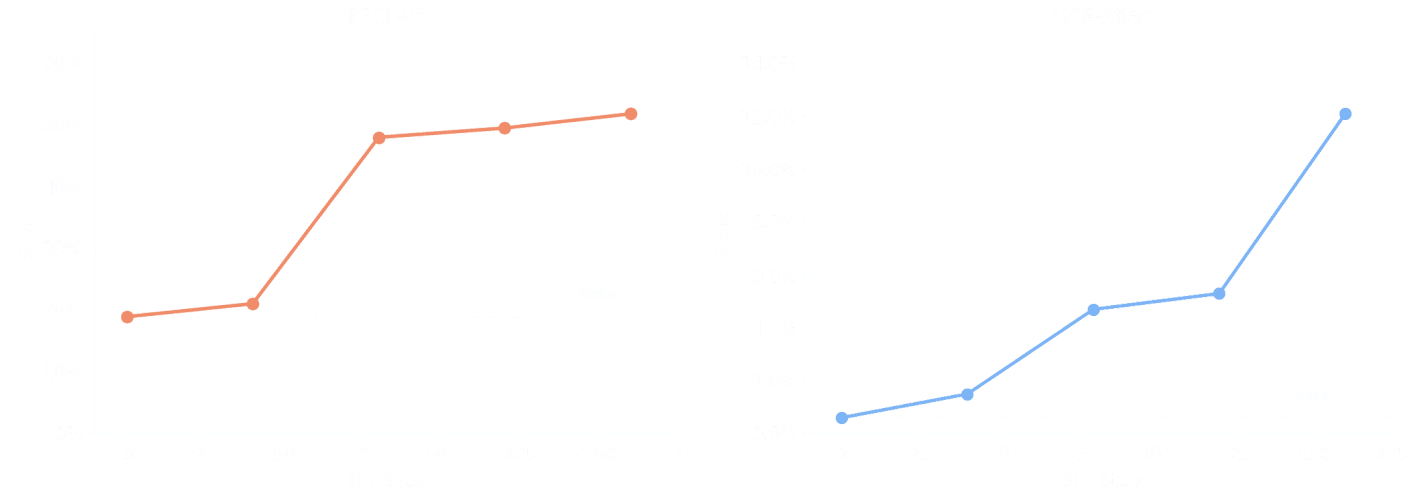
Finally, we fine-tune `Nemotron-3-Nano-30B-A3B-Base-BF16` to test whether training on synthetic tool-calling traces transfers to held-out benchmarks.

Data. We train on 4,417 raw multi-turn conversations with tool calls and tool results from GLM-5.1 on the whole task corpus, without any filtering.

Setup. We trained for 200 steps with a learning rate of 5e-5 (linear decay, 50-step warmup), batch size 8, and 64k context on 16xH200 GPUs. (W&B run)



Results. We evaluate intermediate checkpoints on BFCL-v3 and MCP-Atlas. The base model starts near zero on both benchmarks. SFT on just 4.4k `general-agent` traces lifts BFCL from 18.9% to 52.3% and MCP-Atlas from 0.6% to 12.1% — closing in to the final post-trained model (73.5% / 45.5%) which was trained on orders of magnitude more data.



SFT evals. BFCL-v3 and MCP-Atlas scores at intermediate checkpoints.

Future Work

This work can be seen as a step towards our broader research vision of closing the loop towards self-improving agents via automated environment building. We believe the environment has many of the right ingredients which allow us to execute on this research vision and evolve our tooling and platform towards it:

- training agents, not models (train any task in any harness)
- compose multiple agents (multi-agent episodes like synthesizer-solver, solver-grader, etc.)

Below, we list some of the concrete next steps we believe are crucial to execute on this vision.

Evolve corpus difficulty

stronger gating model — opening room for tasks that challenge frontier models.

Domain Generalization

The `general-agent` environment was built to create a task set with maximally diverse, fully synthetic tools. We believe a very similar recipe can be applied to synthetically generate environments in many other domains, such as terminal-use or document-retrieval, when grounding task generation in real-world seed data.

Multi-agent training

For this version of the environment, the synthesizing loop runs offline to create a fixed task corpus which can then be used for downstream training of a solver agent. However, because both agents are built as verifiers environments, the synthesizing episode — which contains two agents (synthesizer & solver) — itself is trainable. We aim to enable such multi-agent training in the future, allowing us to truly evolve the task corpus during training.

Abstractions

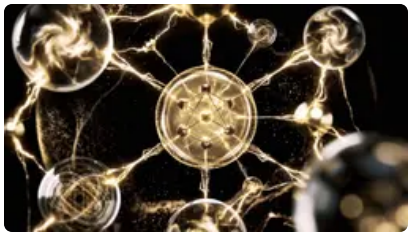
We recently merged a preview of what will become `verifiers v1` — a full decoupling of tasks (what to solve) and harnesses (how to solve) with rich ways to compose both. The `general-agent` environment provides the perfect playground for testing those new abstractions as it requires both local and sandboxed execution, has task-specific custom tools and a multi-agent subtask.

Authors

Mika

Share

Blackwell.



Prime Intellect Joins the NVIDIA Nemotron Coalition to Advance Open Frontier Models

Prime Intellect joins the NVIDIA Nemotron Coalition alongside leading open model builders, contributing post-training infrastructure, 2,500+ RL environments, and a hosted loop to specialize Nemotron for real work.

Releasing Hosted Evaluations: Making benchmarking effortless

We are releasing Hosted Evaluations, giving everyone access to scalable infrastructure to test models on their own evaluations across the hundreds of community-made environments on the Environments Hub.

START TRAINING > BOOK A CALL >

PLATFORM

LAB
COMPUTE
RESEARCH

COMPANY

CAREERS 24
MERCH
CONTACT

COMMUNITY

X
LINKEDIN
DISCORD
LUMA

RESOURCES

DOCS
WRITINGS
EVENTS
MERCH

TERMS

TERMS OF SERVICE
PRIVACY POLICY
SECURITY POLICY